

Knoema Engine

A Lightweight LLM-Based Multi-Agent Social Simulation Runtime

Celovin | hello@celovin.com | April 2026

Abstract

Knoema Engine is an early-stage Python runtime for LLM-based multi-agent social simulation. It combines persona prompts, hierarchical memory, directed social relationships, environment context, PAD emotion state, LLM-backed decision generation, and replayable logs behind a compact API. The MVP demonstrates one shared runtime across persistent-memory game NPCs, fictional public-safety scenario replay, and reproducible academic social simulation.

1. Introduction

LLM-based agents make it practical to represent social behavior as language-rich decisions over memory, goals, and environment state. Knoema Engine targets a narrow engineering goal: a small reusable runtime that can be installed as a Python package, demonstrated in notebooks, inspected through a dashboard, and connected to game engines.

The first implementation focuses on reliable boundaries instead of scale. All demos run without API keys through deterministic local responders, while the same interfaces can be connected to Anthropic or OpenAI clients. Simulation logs export as JSONL so notebooks, dashboards, and adapters consume the same artifact.

2. Related Work

Generative Agents showed that natural-language memory, planning, and reflection can produce believable social behavior in interactive environments. Concordia frames generative social simulation as grounded interactions among entities. AutoGen focuses on conversable LLM agents for task-oriented applications. Mesa remains a strong classical Python baseline for reproducible agent-based modeling.

Knoema differs by keeping a compact engine API and demonstrating the same runtime across games, synthetic replay research, and academic notebooks.

3. Architecture

The core package exposes dataclasses for Personality, Emotion, Memory, Action, and WorldEvent. Higher-level modules compose these types into personas, memory stores, social relationships, environment context, emotion state, and a simulation loop.

Simulation flow: Persona + Memory + Relationship + Environment + Emotion -> DecisionEngine -> LLMGateway -> Action -> Simulator -> JSONL -> Notebooks, Dashboard, Godot Adapter.

Module	Responsibility	MVP status
Persona	Identity, values, goals, prompt rendering	Implemented
Memory	Short-term FIFO and SQLite + FAISS retrieval	Implemented
Relationship	Directed trust, familiarity, and interaction weight	Implemented
Decision	Prompt building and strict JSON action parsing	Implemented
Adapters	Dashboard and Godot scaffold	Implemented

4. Implementation Snapshot

The current repository implements the MVP as an installable Python package named `knoema-engine`. Source modules live under `src/knoema`, tests under `tests`, notebooks under `examples`, game integration under `adapters/godot`, and dashboard code under `dashboard`. The repository also includes dual-language README files, architecture notes, research positioning notes, and GitHub Actions CI.

Area	Files	Verification
Core package	<code>src/knoema/*.py</code>	Unit tests and mypy
Memory	<code>src/knoema/memory</code>	Retrieval and persistence tests
Decision loop	<code>decision.py</code> and <code>llm/</code>	Mock client tests
Simulation	<code>simulator.py</code> and <code>events/</code>	7-day run test
Examples	<code>examples/*.ipynb</code>	nbconvert execution
Dashboard	<code>dashboard/</code>	Pure logic tests and HTTP smoke test

The local verification run before this report passed 52 `pytest` tests, `ruff` check `.`, and `mypy` `src`. The GitHub Actions workflow runs the same lint, type, and test gates on Python 3.11 and 3.12. The public repository surface is scanned to keep private planning documents and unrelated entity references out of committed files.

The recommended live demonstration starts with the first notebook, then opens the dashboard with the bundled sample log, and finally shows the Godot adapter directory. This order keeps the narrative concrete: first define agents, then observe exported traces, then show how the same action surface can be consumed by an external runtime.

The repository is private during preparation and is intended to be made public after the first notebook, Godot adapter, and documentation are stable. Runtime artifacts such as logs, SQLite databases, FAISS indexes, checkpoints, and private planning documents are ignored by `git`.

5. MVP Demonstrations

Demo	Purpose	Verification
Dormitory	Two synthetic students over seven days	672 action records
Fictional replay	Synthetic property incident reconstruction	Milestone coverage above 0.8
Game NPC	Cross-session memory retrieval	Godot-style payload
Dashboard	JSONL inspection	52 tests pass

The notebooks are demonstration artifacts rather than controlled experiments. They verify that the architecture is runnable, inspectable, and safe to show without private data or API credentials.

The dormitory simulation verifies long-running social interaction and relationship updates. The fictional replay notebook verifies trace consistency against process milestones. The game NPC notebook verifies cross-session memory retrieval and adapter-friendly payloads. The dashboard verifies that exported JSONL can be inspected without rerunning the simulator.

6. Safety Boundary and Limitations

The public-safety track is constrained to fictional, synthetic, non-identifying examples. The replay notebook does not estimate risk, identify suspects, or predict future crime. Its milestone coverage metric measures trace consistency against a known synthetic script.

Current limitations include deterministic local responders, a lightweight hash embedding encoder, simple relationship update heuristics, a scaffold-level Godot adapter, and a read-only dashboard.

The next evaluation layer should add model-backed runs, semantic encoder comparisons, larger agent populations, and a review rubric for plausibility, repetitiveness, safety, and trace faithfulness.

7. Conclusion

Knoema Engine demonstrates a compact path from LLM-based social simulation research to game and research tooling. The next steps are larger multi-agent scenarios, stronger semantic retrieval, benchmark scripts, richer game-engine adapters, and more formal evaluation protocols for synthetic replay tasks.

References

- Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442. <https://arxiv.org/abs/2304.03442>
- Vezhnevets, A. S. et al. (2023). Generative Agent-Based Modeling with Actions Grounded in Physical, Social, or Digital Space Using Concordia. arXiv:2312.03664. <https://arxiv.org/abs/2312.03664>
- Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. <https://arxiv.org/abs/2308.08155>
- ter Hoeven, E. et al. (2025). Mesa 3: Agent-Based Modeling with Python in 2025. Journal of Open Source Software, 10(107), 7668. <https://doi.org/10.21105/joss.07668>
- Johnson, J., Douze, M., and Jegou, H. (2017). Billion-Scale Similarity Search with GPUs. arXiv:1702.08734. <https://arxiv.org/abs/1702.08734>